

Switch, Nested If, Nested Loops & Nested Functions

PADRI VJEKO CENTRE TSS - PYTHON PROGRAMMING L5NIT



Group 1 — Switch statement

What it is, syntax, examples & applications

1 What is a switch statement?

A switch statement is a control structure used to execute one block of code among many possible options, depending on the value of a variable or expression. Think of it like a menu: choose an option (case) and the program runs that block.

2 Syntax Example (C / Java / JavaScript)

```
switch (day) {  
  case 1:  
    printf("Monday");  
    break;  
  case 2:  
    printf("Tuesday");  
    break;  
  case 3:  
    printf("Wednesday");  
    break;  
  default:  
    printf("Invalid day");  
}
```

Copy

- The switch checks **day**.
- It matches one of the **case** values.
- The **break** stops checking further cases.
- **default** runs if no case matches.

3 Python (match / case)

Python 3.10+ introduced pattern matching (`match / case`), which works similarly to `switch`.

```
day = int(input("Enter day number (1-3): "))
```

Copy

```
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Invalid day")
```

4 Real-life examples

Scenario	Description	Example
 Day of week	Decide message by day	<code>case 1: print("Workday")</code>
 Menu selection	User picks program option	<code>case 2: show_profile()</code>
 Banking app	Action by transaction type	<code>case 'withdraw': withdraw_money()</code>
 Game	Handle player action	<code>case 'jump': jump()</code>

5 When to use

Use when many conditions test one variable. **Avoid** when conditions are complex or involve multiple variables.

Cleaner than many `if/elif`

Not for ranges



Group 2 — Nested If

If statements inside other ifs (Python examples)

1 What is a Nested If?

An `if` inside another `if`. Useful when a second condition depends on the first.

2 Basic Syntax

```
if condition1:
    if condition2:
        print("Both conditions are true")
    else:
        print("Only condition1 is true")
else:
    print("Condition1 is false")
```

3 Example

```
age = 20
has_id = True
```

Copy

```
if age >= 18:
    if has_id:
        print("You are allowed to enter.")
    else:
        print("You must bring your ID.")
else:
    print("You are too young to enter.")
```

4 Real-life examples

- 🎫 Event entry: If age ≥18 then check ID.
- 🏠 Banking: If account exists then check balance.
- 🔒 Login: If username exists then check password.

5 When to use

Use for related, dependent checks. Avoid deep nesting — prefer `elif` or functions instead.

6 Decision example

```
marks = 80

if marks >= 50:
    if marks >= 75:
        print("Distinction")
    else:
        print("Pass")
else:
    print("Fail")
```



Group 3 — Nested Loop statements

Loops inside loops (Python examples)

1 What is a Nested Loop?

A loop inside another loop. Useful for tables, grids, and repeating inner tasks per outer item.

2 Syntax

```
for i in range(outer_limit):
    for j in range(inner_limit):
        print(i, j)
```

3 Example

```
for student in ["Alice", "Bob"]:
    for subject in ["Math", "Science", "English"]:
        print(f"{student} - {subject}")
```

4 Real-life examples

Scenario	Explanation	Example
 Students' marks	For each student, check each subject	Input marks per student
 Attendance before exam	Check notes then attendance per student	notes + attendance check
 Tables / Grids	Multiplication tables or matrix ops	1×1 to 10×10

5 Example: marks collection

```
students = ["Alice", "Bob"]
subjects = ["Math", "Science", "English"]

for student in students:
    print(f"\nResults for {student}:")
    for subject in subjects:
        marks = int(input(f"Enter {subject} marks: "))
        print(f"{subject}: {marks}")
```

6 Multiplication table

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} × {j} = {i*j}")
    print()
```

Tip: Avoid many nested loops on large data — prefer functions, libraries, or vectorized operations.



Group 4 — Nested Function

Inner functions used as helpers

1 What is a Nested Function?

A function defined inside another function. It helps keep helper logic close and private.

2 Basic Syntax

```
def outer_function():
    print("This is the outer function.")
    def inner_function():
        print("This is the inner function.")
    inner_function()
```

3 Real-life example

```
def check_exam_eligibility(name, theory_marks,
attendance):
    def passed_theory():
        return theory_marks >= 50
    def good_attendance():
        return attendance >= 75
    if passed_theory() and good_attendance():
        print(f"{name} is allowed to attend the
practical exam.")
    else:
        print(f"{name} is NOT allowed to attend the
practical exam.")
```

4 Analogies & applications

-  Exams: small checks inside a main eligibility function
-  Banking: validate account then balance with inner checks

-  E-commerce: compute discount & delivery inside order processing

5 When to use

Use nested functions to organize helper tasks and hide inner logic. If a helper is reused widely, make it a top-level function.

6 Small example

```
def student_report(name):  
    def greeting():  
        print(f"Hello, {name}!")  
    def remarks():  
        print("Keep studying hard and attend all  
practicals.")  
    greeting()  
    remarks()
```

[Back to top](#)